基于Claude+Deepseek+Obsidian的本地知识库搭建

前情提要

这不是传统意义上的"按步骤照做就完事"的教程，更像是我给大家提供一种思路——因为**适合自己的才是最好的**。

本教程面向 **Nuist-FFF 群友**，意在为大家提供 AI 时代下整理笔记的新思路。

读完这篇教程你会得到什么？

- 一个**本地运行的 AI 助手**（用 DeepSeek 的大脑 + Claude Code 的框架）
- 一个**Obsidian 知识库**，所有笔记都在你电脑上，永远属于你
- 一条完整的**知识闭环**：获取信息 → AI 整理 → 存入知识库 → AI 再优化 → 产出新笔记

这个知识库的优缺点

优点

1. **从零开始，无限扩展**：不像 Notion、语雀有存储限制，你的硬盘多大，知识库就能多大
2. **AI 整理效率极高**：传统笔记需要手动排版、分类、加标签；AI 帮你自动完成，你只需要思考和提问
3. **可升级到云端**：后续借助 Git + GitHub，能像游戏云存档一样同步到多台电脑
4. **碾压传统笔记格式**：对文字和代码类笔记，Markdown 比 Word / PPT / 手写笔记强一个维度——纯文本、可搜索、可版本管理
5. **形成完整知识生态**：

```
你与AI对话 → 产出文本
网上获取的资料 → 转为Markdown
        ↓
    存入Obsidian知识库
        ↓
    AI整理、补充、规范
        ↓
    产出新笔记 → 再经AI优化 → 回到知识库
        ↓
      形成闭环 ✅
```

缺点

- **对手写笔记不友好**：已有的纸质笔记需要手动录入或拍照 OCR，这一步比较费劲
- **需要一点学习成本**：下面会讲到前置知识，但请放心——这些都是你迟早要学、学完受益终身的东西

前置知识（零基础友好版，每个都给你讲明白）

> **先打一剂强心针**：你可能会想「我就写个笔记，怎么要学这么多？太麻烦了，不搞了。」
>
> 但事实是：下面这些知识几乎是一个程序员**吃饭的家伙**。哪怕你不是程序员，只要学过一门编程语言，你一定听说过它们。这些知识**不是只对这个项目有用**——学了它们，等于学了编程界的加减乘除。

---

1. Markdown 是什么？

一句话：一种用纯文本写出带格式文章的写法。

对比	TXT	Word (docx)	Markdown
能不能加粗、标题?	❌	✅	✅
文件大小	很小	大	很小
能不能用 Git 管理?	✅	❌ Git 看不懂	✅
跨平台打开	✅ 任何设备	需要 Office	✅ 任何设备
学习成本	零	中等	**10 分钟**

举个例子，下面这段纯文本：

```
# 这是一级标题
## 这是二级标题

**这是加粗文字**

- 列表项1
- 列表项2

[这是链接](https://www.baidu.com)
```

渲染出来就是带格式的排版。你接下来写的所有笔记都是 Markdown 格式，Obsidian 就是帮你编辑、预览、管理 Markdown 文件的神器。

2. API 和 URL 是什么?

> 为什么我们可以在 Claude Code 框架下使用 DeepSeek 作为大脑?

URL：就是你浏览器地址栏里的网址，比如 `https://api.deepseek.com`。它就是互联网上一个「门牌号」。

API：全称「应用程序接口」。把它理解成一个**餐厅的服务员**：

- 你（Claude Code）想点菜（向 DeepSeek 提问）
- 你不能直接进厨房（不能直接操作 DeepSeek 的服务器）
- 你需要通过服务员（API）来传话
- 服务员需要验证你有钱付账（API Key / Token）

在这个教程里：你要去 DeepSeek 买的「API Key」= **吃饭凭证**；你要在 CCSwitch 里填的「URL」= **餐厅地址**。

那为什么可以在 Claude Code 框架下用 DeepSeek?
Claude Code 本质上是一个「对话框」，它不挑对话对象——只要对方提供符合标准的 API，它就能对话。CCSwitch 就是一个「转接头」，把 Claude Code 默认发往 Anthropic 的请求截住，转发给 DeepSeek。

---

3. Git 是什么?

一句话：代码/文本的「游戏存档系统」。

玩过单机游戏吗？核心机制就是**存档**：

你想做什么	Git 命令	游戏类比
保存当前进度	`git commit`	存个档
改坏了想回退	`git reset`	读档
查看改过多少次	`git log`	查看存档记录
多人协作	`git merge`	合并不同人的存档

GitHub 是什么? Git 管本地存档，GitHub 管云存档。你在本机 `git push`，存档就上传到 GitHub；换台电脑 `git pull`，存档就下载回来。

本教程中 Git 的作用：把你的 Obsidian 笔记自动备份到 GitHub，防止电脑坏了笔记全丢。

> 如果你是一个程序员，却不知道 Git 是什么，那就像司机不知道方向盘是什么——这几乎是所有程序员每天用的基础设施。

---

4. Node.js 是什么？

一句话：让 JavaScript（网页脚本语言）能在你电脑上脱离浏览器独立运行。

本来 JavaScript 只能活在浏览器里。Node.js 把它放了出来，让它能像 Python 一样在你的 Windows/Mac 上直接跑。

前端 vs 后端 vs 桌面端：

- **JavaScript（前端）**：在浏览器里运行，管网页的交互效果
- **Node.js（后端/桌面端）**：在操作系统上独立运行，能读写文件、访问网络、跑命令行工具
- Claude Code 就是用 Node.js 写的，所以**你必须先装 Node.js 才能装 Claude Code**

> 类比：Node.js 之于 JavaScript，就像 JVM 之于 Java——它是运行环境。

---

5. Obsidian 是什么？

一句话：基于本地 Markdown 文件的笔记软件，你的「第二大脑」。

跟 Notion、语雀最大的区别：**你的数据 100% 在你电脑上**，不是存在别人的服务器里。
Obsidian 不联网也能用，你的笔记就是电脑上实实在在的 `.md` 文件，用任何编辑器都能打开。

核心功能：

- **双向链接**：用 `[[笔记名]]` 把笔记关联起来，形成知识网络
- **图谱视图**：可视化看到所有笔记之间的关联
- **插件生态**：大量社区插件扩展功能

搭建知识库，你现在需要什么

序号	需要的东西	说明
1	一台 Windows 10/11 电脑	Mac 也行，但本教程以 Windows 为例
2	可以访问外网的网络环境	下载 GitHub、注册等需要
3	一双手	用来敲键盘

序号	需要的东西	说明
4	大约 2 小时的时间	第一次配置会慢一点，之后一劳永逸
5	10～20 元人民币	DeepSeek API 的最低充值金额，能用很久

第一步：下载安装 Git

Git 是后面所有操作的基础，我们先把 Git 装好。

1.1 下载 Git

1. 打开你的浏览器（Edge、Chrome 都行）
2. 在地址栏输入 **https://git-scm.com** ，回车
3. 页面加载后，右侧会有一个硕大的按钮写着 `Download for Windows`，**点击它**
   - 按钮上会显示最新版本号，比如 `2.47.0`
4. 浏览器会开始下载，文件大约 60MB，等它下载完
5. 下载完成后，你会在浏览器底部或右上角看到下载完成的提示

> **如果官网访问很慢或打不开**：在百度搜索「Git 国内镜像」，用淘宝镜像站（npm.taobao.org）下载。

1.2 安装 Git（每一步都请仔细看，别点错）

1. 找到你刚下载的文件，文件名类似 `Git-2.47.0-64-bit.exe`，**双击打开**
2. **第1屏 - 许可协议**：直接点 `Next`
3. **第2屏 - 选择安装路径**：
   - **强烈建议不要改，保持默认** `C:\Program Files\Git`
   - 如果你想换位置，记住**路径里不能有中文**，否则后面会出问题
   - 点 `Next`
4. **第3屏 - 选择组件**：
   - 保持默认勾选不动，点 `Next`
   - 确保 `Git Bash` 和 `Git GUI` 前面有勾
5. **第4屏 - 选择开始菜单文件夹**：直接点 `Next`
6. **第5屏 - 选择默认编辑器**：⚠️ **这一屏很重要！**
   - 默认选中的是 `Vim`——这是一个上古编辑器，操作方式极其反人类，你不会想用的
   - **点击下拉菜单，选择** `Use Notepad as Git's default editor`（用记事本当默认编辑器）

- 如果你电脑装了 VS Code，也可以选 `Use Visual Studio Code as Git's default editor`
- 选好后点 `Next`

7. **第6屏 - 调整分支名称**：
   - 选择 `Let Git decide`（默认选项），不要改，点 `Next`
8. **第7屏 - 调整 PATH 环境**：⚠️ **这一屏很重要！**
   - 三个选项，选择**第二个**：`Git from the command line and also from 3rd-party software`
   - 选这个才能让你在 cmd 里直接敲 `git` 命令
   - 点 `Next`
9. **第8屏 - 选择 SSH 客户端**：
   - 选择 `Use bundled OpenSSH`（默认），点 `Next`
10. **第9屏 - 选择 HTTPS 传输后端**：
    - 选择 `Use the OpenSSL library`（默认），点 `Next`
11. **第10屏 - 配置行尾符转换**：
    - 选择**第一个**：`Checkout Windows-style, commit Unix-style line endings`
    - 点 `Next`
12. **第11屏 - 选择终端模拟器**：
    - 选择 `Use MinTTY`（默认），点 `Next`
13. **第12屏 - 默认行为**：
    - 选择 `Default (fast-forward or merge)`（默认），点 `Next`
14. **第13屏 - 选择凭据管理器**：
    - 选择 `Git Credential Manager`（默认），点 `Next`
15. **第14屏 - 额外选项**：
    - 默认勾选即可，不用改，点 `Next`
16. **第15屏 - 实验性选项**：
    - **全部不勾**，点 `Install`
17. 等待蓝色进度条跑完，点 `Finish`

1.3 验证 Git 是否安装成功

1. 按键盘上的 `Win` **键**（Windows 徽标键，在 Ctrl 和 Alt 之间）+ `R` **键**
2. 会弹出「运行」窗口，输入 `cmd`，然后按回车
3. 出现一个黑底白字的窗口——这就是**命令提示符**（后面我们简称 **cmd**）
4. 在 cmd 里输入以下命令，然后按回车：

```
git --version
```

5. 如果看到类似 `git version 2.47.0.windows.1` 这样的输出 → **安装成功！** ✅
6. 如果看到「不是内部或外部命令」→ 说明安装有问题，回头检查第 7 屏 PATH 是不是选错了

1.4 配置你的 Git 身份

装好 Git 后，告诉它「你是谁」。在 cmd 里依次输入下面两行（把名字和邮箱换成你自己的）：

```
git config --global user.name "你的名字拼音"
git config --global user.email "你的邮箱@example.com"
```

每输完一行按一次回车。建议用户名用**拼音**，不要用中文，避免编码问题。

第二步：注册 GitHub 账号

2.1 注册

1. 打开浏览器，地址栏输入**https://github.com
2. 页面加载后，点击右上角的 `Sign up` 按钮
3. 输入你的**邮箱地址**，点击 `Continue`
4. 设置一个**密码**（至少 8 位，要包含数字和字母），点击 `Continue`
5. 输入一个**用户名**（这是你的唯一标识，以后你的主页就是 `github.com/你的用户名`），点击 `Continue`
6. 问你是否要接收产品更新邮件，输入 `y` 或 `n` 都行，点击 `Continue`
7. 完成**人机验证**：按页面提示识别图片中的物体（比如「点击所有包含自行车的图片」）
8. 点击绿色的 `Create account` 按钮
9. GitHub 会向你的邮箱发一封验证邮件，去邮箱查收，把验证码输入到页面上
10. 验证成功后，会跳到一个个性化设置页面——**全部跳过**，点最底下的 `Skip personalization`
11. 注册完成！

2.2 创建仓库（放你的笔记）

1. 登录 GitHub 后，点击页面**右上角的 `+` 号**，在下拉菜单里选 `New repository`
2. 在 `Repository name` 里输入仓库名，比如 `my-obsidian-notes`
3. `Description` 可填可不填，写一句描述比如「我的个人知识库」
4. 选择 `Private`（私密仓库——只有你自己能看到笔记内容）
5. **不要勾选** `Add a README file`（我们后面自己来）
6. 点击页面底部绿色的 `Create repository` 按钮
7. 创建完成后页面会显示一堆命令行指令，**先不管它**。你只需要记住你的仓库地址，格式是：

```
https://github.com/你的用户名/my-obsidian-notes.git
```

把这个地址复制到一个记事本里存起来，后面要用。

第三步：下载安装 Obsidian

3.1 下载 Obsidian

1. 打开浏览器，地址栏输入 **https://obsidian.md
2. 页面中间有一个大大的 `Download` 按钮，点击它
3. 网站会自动识别你是 Windows，显示 `Download for Windows`，点击下载
4. 等待下载完成，文件大约 100MB

3.2 安装 Obsidian

1. 找到下载好的文件（`Obsidian-x.xx.x.exe`），双击打开
2. 安装程序会自动完成——不需要你选路径，Obsidian 会装到用户目录下
3. 安装完会自动打开 Obsidian

3.3 创建你的第一个 Vault（知识库）

Obsidian 把每个知识库叫做一个 **Vault**——可以理解为「一个项目文件夹」。

1. Obsidian 打开后，你会看到一个 Vault 管理界面
2. 点击 `Create` 按钮（创建新 Vault）
3. `Vault name`：输入名字，比如 `MyNotes`
4. `Location`：点击 `Browse` 按钮，选择一个位置。**建议在 D 盘创建一个专门放笔记的文件夹**，比如 `D:\MyObsidianVault`
5. 点击 `Create`
6. 创建完成！你会看到 Obsidian 主界面——左边是文件列表，中间是编辑器

3.4 推荐视频

如果你想深入了解 Obsidian 的基本操作（编辑、双向链接、图谱视图、Git 集成），推荐看这个视频：

Obsidian 基础 + Git 集成教程

> 视频里的 AI 部分用的是 Gemini CLI，比较鸡肋，也不通用。你正在搭建的 **Claude Code + DeepSeek** 方案要强得多，能广泛用于代码、闲聊、对话、笔记等各种场景。Git 和 GitHub

> 的部分如果你已经在用就不需要看。

第四步：下载安装 Node.js

> 为什么需要 Node.js？因为 Claude Code 是用 Node.js 写的。你必须先装 Node.js，后面才能用 `npm` 命令安装 Claude Code。

4.1 下载 Node.js

1. 打开浏览器，地址栏输入 https://nodejs.org，回车
2. 页面中间有两个绿色大按钮：
   - 左边 **LTS** = 长期支持版，
   - 右边 **Current** = 最新版，
   - 推荐最新版，因为claude对node.js版本有要求
3. 浏览器开始下载，文件大约 30MB，等待完成

4.2 安装 Node.js

1. 找到下载好的文件（`node-v22.xx.x-x64.msi`），双击打开
2. 点击 `Next`
3. 勾选 `I accept the terms in the License Agreement`，点 `Next`
4. **选择安装路径**：保持默认，直接点 `Next`
5. **自定义安装**：保持默认，所有组件前面都有勾就行。**特别注意：** `npm package manager` **前面必须是勾上的**——这是安装 Claude Code 的关键工具。点 `Next`
6. **工具安装选项**：会问你要不要装额外工具——**不需要勾选** `Automatically install the necessary tools`，这是给 C++ 开发者用的。点 `Next`
7. 点 `Install`，等待进度条跑完
8. 点 `Finish`

4.3 验证 Node.js 是否安装成功

1. 按 `Win + R`，输入 `cmd`，回车打开命令提示符
2. 输入下面两行命令（每输完一行按一次回车）：

```
node --version
npm --version
```

3. 如果分别看到类似 `v22.xx.x` 和 `10.x.x` 这样的版本号 → **安装成功！** ✅

> 提示： `npm` 全称 Node Package Manager，是 Node.js 自带的包管理工具。你后面的 `npm install` 命令就是用它在网上自动下载安装各种工具。

第五步：安装 Claude Code

Claude Code 是 Anthropic（Claude 的开发商）官方出的命令行 AI 助手。我们先把它装上，后面再给它换个国产大脑。

5.1 用 npm 安装 Claude Code

1. 按 `Win + R`，输入 `cmd`，回车打开命令提示符
2. 在 cmd 里输入下面这行命令，然后回车：

```
npm install -g @anthropic-ai/claude-code
```

3. 你会看到一行行滚动的下载信息——这是 npm 在自动下载 Claude Code 以及它依赖的各种包
4. 等待 1～3 分钟（取决于你的网速），直到命令行重新出现闪动的光标，说明安装完毕

> `-g` **是什么意思？** 它表示「全局安装」——把 Claude Code 装成系统级命令，让你在任何目录下敲 `claude` 都能启动它。

5.2 验证安装

1. 在 cmd 里输入：

```
claude --version
```

2. 看到版本号 → 安装成功！
3. 再输入 `claude` 并回车，启动看看：

```
claude
```

4. 它会启动，但马上提示需要登录或 API Key——**不用管！直接按键盘上的 `Ctrl + C`（同时按住 Ctrl 键和 C 键）退出**
5. 能启动并提示需要 API → Claude Code 本身安装成功了。下一步我们就给它配上 DeepSeek 的 API。

> **发生了什么？** Claude Code 默认要去连接 Anthropic（Claude 的开发商）的服务器。但我们不打算花美金用 Claude——我们要给它换成便宜好用的 DeepSeek。这事 CCSwitch 来做。

第六步：注册 DeepSeek 并获取 API Key

6.1 注册 DeepSeek 账号

1. 打开浏览器，地址栏输入 https://platform.deepseek.com，回车
2. 点击右上角的「注册」
3. 用手机号注册最简单，也可以用邮箱
4. 按页面提示完成注册和登录

6.2 充值和创建 API Key

> **为什么要充值？** Claude Code 每次对话都会调用 DeepSeek 的 API，API 按使用量收费。但好消息是：DeepSeek 非常便宜，最低充 10 元就够你用很久很久。

1. 登录 DeepSeek 开放平台后，点左侧菜单的「充值」或「费用中心」
2. 选择充值金额（**最低 10 元就行**），微信或支付宝扫一扫付钱
3. 充好后，点左侧菜单的 **「API Keys」**
4. 点 **「创建 API Key」** 按钮
5. 起个名字，比如「我的知识库」
6. 点「创建」，页面上会出现一个长长的字符串（以 `sk-` 开头）
7. ⚠️ **立刻复制并保存！** 这个 Key **只显示这一次**，关掉页面后就再也看不到了
8. 把它粘贴到一个记事本里暂时存着，后面要用

> **目前 DeepSeek 有大幅折扣**（截至 2026 年 5 月 31 日），token 消耗价格很低，是入手的好时机。

6.3 记下 API 地址

你的 API Key 需要配合下面这个 URL 使用：

```
https://api.deepseek.com
```

把它也记在记事本里，下一步填到 CCSwitch 里。

第七步：下载配置 CCSwitch

7.1 CCSwitch 是什么？

Claude Code 默认只认 Anthropic 的 API，不认 DeepSeek。CCSwitch 是一个**转接头**——它会在你电脑上启动一个本地代理，把 Claude Code 发出的请求拦截下来，翻译一下，再转发给 DeepSeek。对 Claude Code 来说，它以为自己在跟 Claude 对话；实际上后台跑的是 DeepSeek。

7.2 下载 CCSwitch

1. 打开浏览器，访问 CCSwitch 的 GitHub 页面：https://github.com/taoyi107/CCSwitch
2. 在页面右侧找到 `Releases`（发行版），点进去
3. 在最新版本（标着 `Latest` 的那个）下面，找到 Windows 安装文件，比如 `CCSwitch-Setup.exe`
4. 点击下载
5. 下载完成后双击安装，一路点 `Next` 就行

> 如果 GitHub 访问很慢，可以在群里找我要安装包，或者用 GitHub 加速下载服务。

7.3 配置 CCSwitch

1. 打开 CCSwitch 软件
2. 你会看到一个配置界面，有几个输入框需要填：

配置项	填什么	说明
API URL	`https://api.deepseek.com/v1`	DeepSeek 的接口地址，**注意末尾的 `/v1` 不能漏**
API Key	`sk-xxxxxxxxxxx`	在第六步保存的那个以 `sk-` 开头的字符串
Model	`deepseek-chat`	使用的模型名称

3. **设置模型映射**（这一步非常关键！）：
   - Claude Code 内部用的是 Claude 的模型名称，比如 `claude-sonnet-4-20250514`
   - 但 DeepSeek 不叫这个名字，它叫 `deepseek-chat`
   - 所以需要告诉 CCSwitch：「当 Claude Code 说要找 `claude-sonnet-4-20250514` 时，你帮我去找 `deepseek-chat`」
   - 在 CCSwitch 界面找到「模型映射」（Model Mapping）区域
   - 添加一条规则：**源模型** 填 `claude-sonnet-4-20250514`，**目标模型** 填 `deepseek-chat`
4. 点「保存」，然后点「启动」让 CCSwitch 开始工作
5. **不要关掉 CCSwitch**，让它一直运行在后台（可以在任务栏右下角看到图标）

第八步：验证一切是否正常

8.1 启动 Claude Code

1. 确认 CCSwitch 在后台运行中（任务栏右下角能看到它的图标）
2. 按 `Win + R`，输入 `cmd`，回车
3. 输入 `claude`，回车：

```
claude
```

4. Claude Code 启动了！这次它应该不会再报错要 API，而是正常进入对话界面

8.2 验证模型身份

在 Claude Code 里输入：

```
你是什么模型？
```

如果它回答自己是 **DeepSeek** 的某个模型 → 🎉 **配置成功了！**



> 上图显示的效果是我加了插件后的界面，首次安装没有下方的那些 LOGO，这是正常的。

8.3 再测试一个知识性问题

继续输入：

```
请用简单的话解释什么是 HTTPS
```

如果得到了流畅的中文回答 → 一切就绪，可以开始干活了！

第九步：让 Claude Code 在 Obsidian Vault 里工作

9.1 配置 CLAUDE.md（告诉 AI 怎么做）

`CLAUDE.md` 是 Claude Code 的「工作手册」——每次在项目目录启动 Claude Code，它会自动读这个文件，了解你的要求和规则。

在你的 Obsidian Vault 根目录下（就是 `D:\MyObsidianVault`）新建一个文件叫 `CLAUDE.md`，内容如下：

```
# Obsidian 知识库操作规范

## 写作策略
- 所有笔记使用 Markdown 格式
- 中文和英文之间自动加空格
- 使用 [[笔记名]] 进行双向链接
- 新建笔记前确认不跟已有笔记重复

## 禁止事项
- 禁止修改 .obsidian 目录下的配置文件
- 禁止随意删除已有笔记
- 操作文件前先确认内容
```

```
这里是我自己在用的claude.md里的内容,是让AI根据我的需求自定义写的,可以节省token,防止大量非缓冲写入

# Obsidian 知识库操作规范



## 搜索策略



### Grep 三原则
```

1. **定位用 `files_with_matches`，不用 `content`**

   - 正确：先 `Grep pattern="关键词" output_mode="files_with_matches"` 找到哪些文件相关

   - 错误：直接 `output_mode="content"` 把匹配行全量灌入上下文

2. **必须用 `content` 时加 `head_limit`**

   - `Grep pattern="...\" output_mode="content" head_limit=20`

3. **只在文件移动/重命名后做全量链接校验，内容补充类改动不做**

Read 原则

- **始终带 `offset` + `limit`**，只读需要确认的那几行

- 补充现有笔记时：读 TOC（前 30 行）+ 插入点前后 5 行，不读全篇

- 不为了"对比确认"先把整篇读完再动手——Read 的每一行都算 token

写作策略

- 多文件并行写作用一个消息里的多个 Write/Edit 调用

- 新建笔记时内容一次性写完，不反复 Edit 同一个文件

- Edit 的 `old_string` 尽量短且唯一，避免因匹配不唯一重试

- **同一文件的多处 Edit 合并到一个调用**：改版本号、改目录、改正文尽量一次搞定，不要拆成 3 次 Edit

- **TodoWrite 只在任务开始和全部完成时用**，中间步骤不更新（每次 TodoWrite 重发整个列表）

禁止事项

```
- 禁止全局 `Grep output_mode="content"` 不做 `head_limit`

- 禁止 Read 不加 offset/limit 读长文件

- 禁止在内容补充后做全库链接验证

- **禁止同一文件连续多次 Edit**（应合并为一次或最多两次）

- **禁止用 `Grep content` 去"看看某文件里有没有某内容"**——用 `files_with_matches` 定位文件，再 Read 关键行

- **禁止 TodoWrite 频繁更新**——开始设一次，结束设一次，中间不碰
```

9.2 日常使用流程

以后每次要在这个 Vault 里用 AI：

1. 按 `Win + R`，输入 `cmd`，回车打开命令提示符
2. 切换到你 Vault 的目录：

```
cd D:\MyObsidianVault
(仅仅作为示例,你也可以直接在你Vault的文件夹右键打开cmd,然后输入claude)
```

4. 启动 Claude Code：

```
claude
```

5. 开始跟 AI 对话，让它帮你整理笔记、搜索内容、补充知识
6. 用完了输入 `/exit` 退出

进阶：用 Git 备份知识库到 GitHub

> 你的笔记都在本机，万一硬盘坏了、电脑丢了，笔记全没。用 Git + GitHub 做备份，就像玩游戏开了**云存档**——换台电脑随时同步回来。

操作步骤

1. 打开 cmd，进入你的 Vault 目录：

```
cd D:\MyObsidianVault
```

2. 初始化 Git：

```
git init
```

这会在你的 Vault 文件夹里创建一个隐藏的 `.git` 目录——Git 的所有存档数据都存在这里。

3. 创建 `.gitignore` 文件

有些文件不需要备份（比如 Obsidian 的临时文件、垃圾桶），我们告诉 Git 忽略它们。

在 Vault 根目录新建一个文件叫 `.gitignore`（注意文件名最前面有个点），内容如下：

```
.obsidian/workspace.json
.obsidian/workspace-mobile.json
.trash/
.DS_Store
```

4. 关联你的 GitHub 仓库：

把下面命令里的「你的用户名」换成你在 GitHub 注册的用户名：

```
git remote add origin https://github.com/你的用户名/my-obsidian-notes.git
```

5. 第一次提交和推送：

依次输入以下命令，每行按一次回车：

```
git add .
git commit -m "初始化知识库"
git branch -M main
git push -u origin main
```

- `git add .`：把 Vault 里所有文件加入暂存区（准备存档）
- `git commit -m "xxx"`：创建一个存档，`-m` 后面是存档备注
- `git push`：把存档上传到 GitHub

推送时可能会弹出一个窗口让你登录 GitHub，正常输入账号密码就行。

6. 以后每次更新笔记后：

```
git add .
git commit -m "更新笔记"
git push
```

> **懒人方案**：不想每次手动敲命令？可以在 Obsidian 里安装 **Obsidian Git** 插件，设置每隔 30 分钟自动备份一次。具体怎么装插件可以问群里的 Obsidian 用户。

常见问题 FAQ

Q1: 运行 `node --version` 提示「不是内部或外部命令」

答： Node.js 可能没装好或 PATH 没设对。

1. 去控制面板 → 程序和功能，看看有没有 Node.js，没有就重新装
2. 装了的话，**关掉 cmd 窗口重新打开**再试一次
3. 还不行就重装 Node.js，安装时每一步保持默认选项不动

Q2: `npm install -g @anthropic-ai/claude-code` 报一堆错

答： 大概率是网络问题。

- 在 cmd 里先执行这个命令换到淘宝镜像源：

  ```
  npm config set registry https://registry.npmmirror.com
  ```

- 然后再执行安装命令

如果是权限问题，右键 cmd →「以管理员身份运行」，再执行安装。

Q3: 启动 Claude Code 后连不上、没反应

答： 逐项排查：

1. CCSwitch 有没有在后台运行？（看任务栏右下角）
2. CCSwitch 里的 **API URL 有没有写错？**（注意末尾 `/v1` 不能少）
3. API Key 有没有多复制了空格？直接从记事本粘贴过来
4. DeepSeek 账户余额还有没有？

Q4: AI 回答的质量不如预期

答：

- DeepSeek 和 Claude 原生模型有差距，但日常笔记整理完全够用，性价比极高

- 可以调整 Vault 根目录的 `CLAUDE.md`，修改里面的写作规范来改变 AI 风格
- 提问时尽量具体明确，AI 的回答会更好

Q5: 和直接用 DeepSeek 网页版有什么区别?

对比	DeepSeek 网页版	本方案
操作方式	打开浏览器、复制粘贴	命令行里直接对话
读写文件	不能读写你的本地文件	AI 可以直接操作你的笔记
项目上下文	每次都从头说	自动读取整个 Vault 的文件
高级功能	基础对话	规划模式、文件编辑、项目感知等

Q6: 能换其他国产模型吗（通义千问等）？

答： 完全可以。CCSwitch 支持多个模型提供商，只要目标模型提供了兼容Claude格式的API，填上对应的 URL 和 Key 就能用。通义千问、智谱 GLM、百川等都可以。

Q7: 我不想用 Git 和 GitHub，可以吗?

答： 可以，完全不影响 AI 功能。但**强烈建议至少定期用 U 盘或网盘手动备份**你的 Vault 文件夹——数据无价，丢过一次就懂了。

总结

恭喜！到此为止，你已经拥有了：

- ✅ 一个本地的 Obsidian 知识库
- ✅ 一个能对话的 AI 助手（Claude Code + DeepSeek）
- ✅ Git + GitHub 版本管理和云备份
- ✅ 一个完整的知识管理闭环

接下来做什么?

1. 把手头的学习资料整理成 Markdown 笔记丢进 Obsidian
2. 在你 Vault 目录下启动 `claude`，让 AI 帮你补充、整理、关联笔记
3. 定期 `git push` 备份到 GitHub
4. **享受知识积累的复利效应** 📈

尾言:新手使用API初期要格外关注API的花销,以防你的配置错误导致了AI花销过大,按理说,一整次笔记整理只需要几角钱以内,笔记很少可能一角钱左右,如果花销异常,可以在你和AI的对话窗口追

问AI为什么非缓冲输入巨大,消耗tokens量异常多

另外:对于熟练的AI使用者,一天用十块钱是常态,但关键在于这10块钱创造了什么价值,他是否值得. 比如国内某清华实验室,使用claude的opus4.6模型,一天花销高达5000人民币,至于值不值得,只有使用者自己才知道